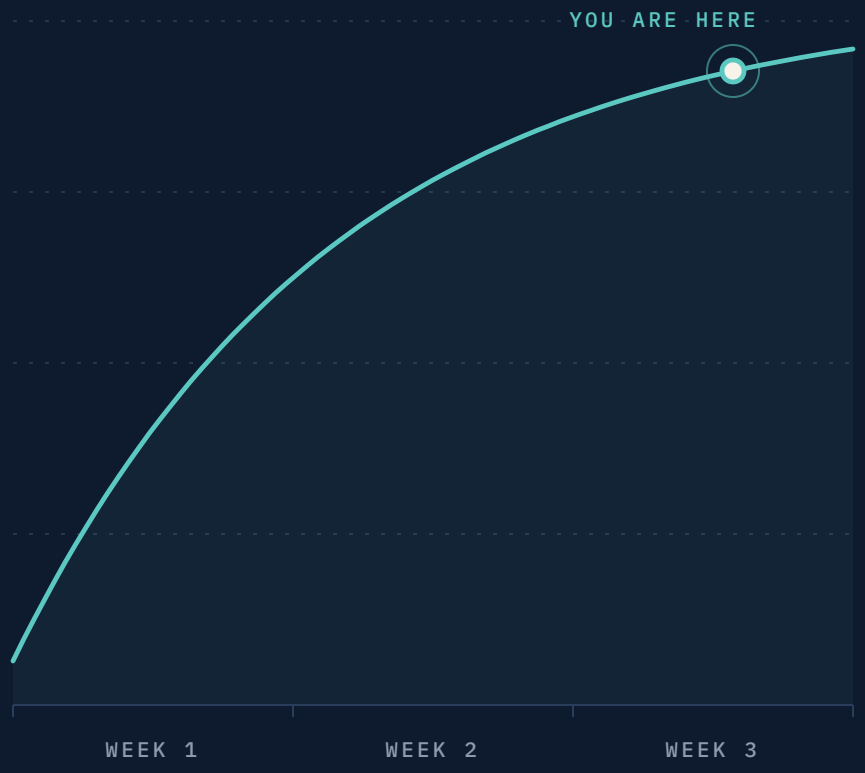
WEEK 3 · DAY 3 · OVERVIEW

Working With an AI Assistant



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 9 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Note

A colleague has been using an AI assistant and forwards what it wrote, each piece signed off "this passes your test": three pandas functions, a 2s10s chart for the morning note, a formula for "percent of days at or below", and a DV01 function. It also recommends a package. Before any of it reaches the desk: tell me which pieces are right, each with a test you ran yourself. Write down what we may paste when we ask it for help. Give marketdata one function that checks a reply we asked for as JSON. Last, it offered a patch to our marketdata.py: do we take it?

Your Route Through Day 3

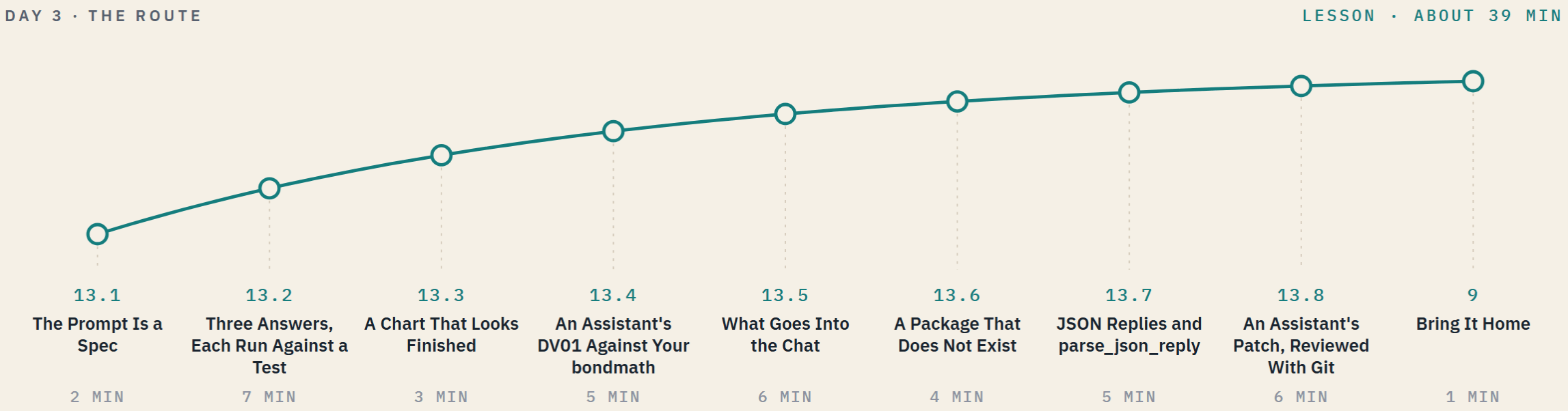


FIGURE 1 Nine sections, in order, with about how long each takes.

Time. About 39 minutes for the lesson, then about 12 for the four exercises.

13.1 The Prompt Is a Spec

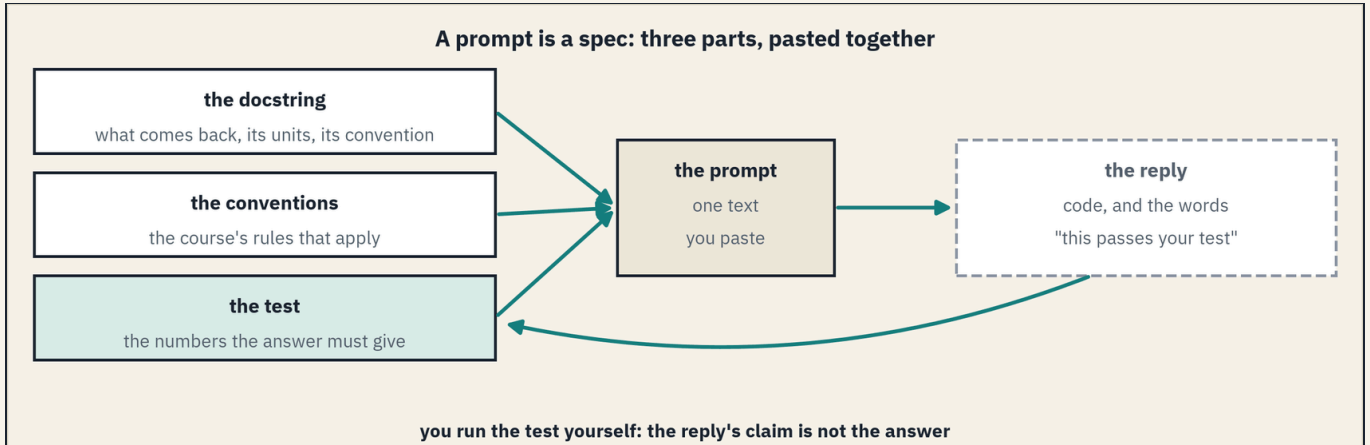


FIGURE 2 A prompt joins the docstring, the conventions, and the test; you run the test, not the assistant.

THE ASSEMBLED PROMPT

```
Write a Python function with pandas only.

month_end_levels(series): the last observation of each calendar month, labelled by
```

The prompt decides every choice in writing, and its test is the review you run yourself.

13.2 Three Answers, Each Run Against a Test

ANSWER A AGAINST THE TEST

```
result = month_end_levels(curve['10 Yr'].loc['2026'])
january = result.index[0]
```

```
AssertionError: January is labelled 2026-01-31
```

ANSWER B AGAINST EXCEL AND YOUR FUNCTIONS

```
answer_b = slope_and_change(curve)
gap_excel = (slope_2026_b - reference['slope_2s10s_bp']).abs().max()
assert gap_excel <= 1e-9 and gap_mine <= 1e-9
```

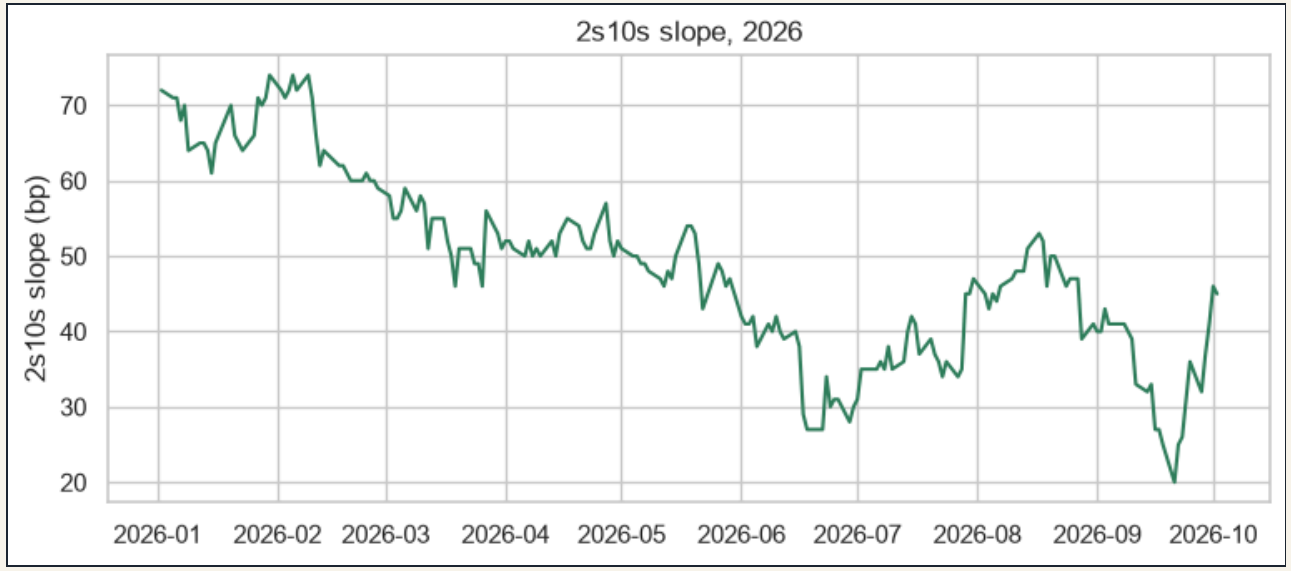
```
largest gap: 0.0e+00 bp from Excel, 0.0e+00 bp from change_bp
```

EXCEL SIDE BY SIDE

JOB	EXCEL
2s10s in bp	`=(D262-B262)*100`, filled down

Not every answer is wrong: the test tells you which to keep.

13.3 A Chart That Looks Finished



READ THE CHART OBJECT

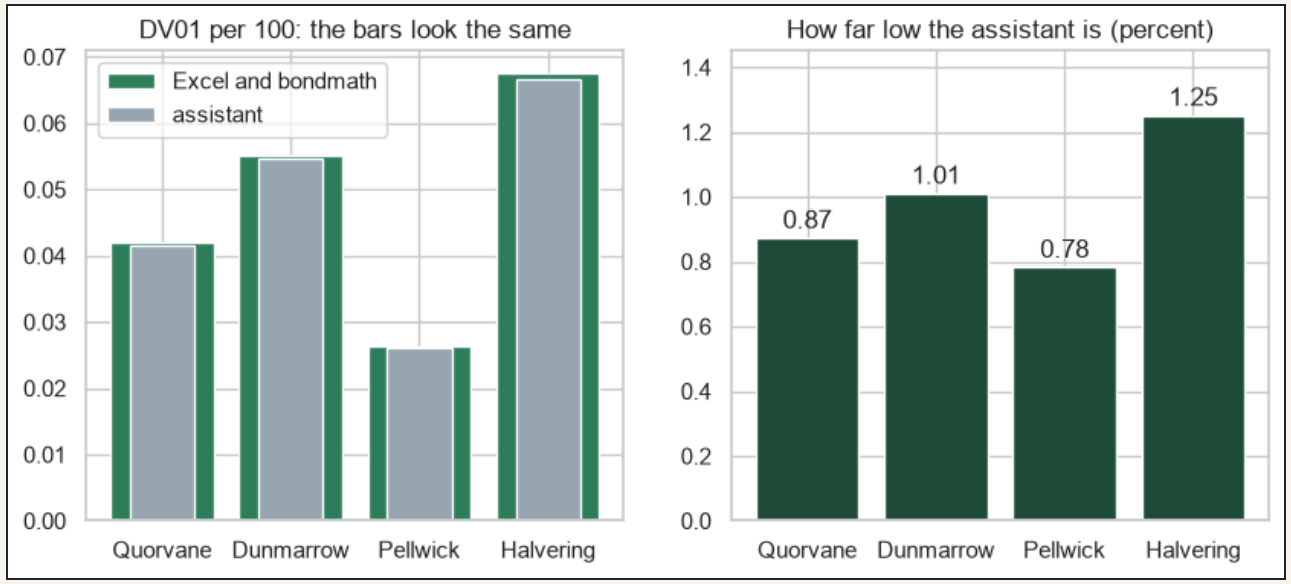
```
label = ax.get_ylabel()
drawn = ax.lines[0].get_ydata()
```

FIX THE LABEL, CHECK AGAIN

```
fig, ax = plot_2s10s(curve.loc['2026'])
ax.set_ylabel('2s10s slope (bp)')
check_chart(ax, slope_2026, 'bp')
```

A chart cannot raise an error for a wrong word: read its label and its line as data, and "looks right" becomes a test.

13.4 An Assistant's DV01 Against Your bondmath



EXCEL SIDE BY SIDE

JOB	EXCEL
DV01 per 100, Course 2's cell	`=B4*(B1+B2)/10000`
The assistant's DV01	`=B4*B1/10000`

COURSE 2'S FOUR BONDS

	ISSUER	EXCEL_DV01
0	Quorvane Energy	0.041903
1	Dunmarrow Rail	0.055175
2	Pellwick Paper	0.026359
3	Halvering Foods	0.067511

The assistant's DV01 is about 1 percent low on every bond, and only a test between coupons, against a reference you trust, shows it.

13.5 What Goes Into the Chat

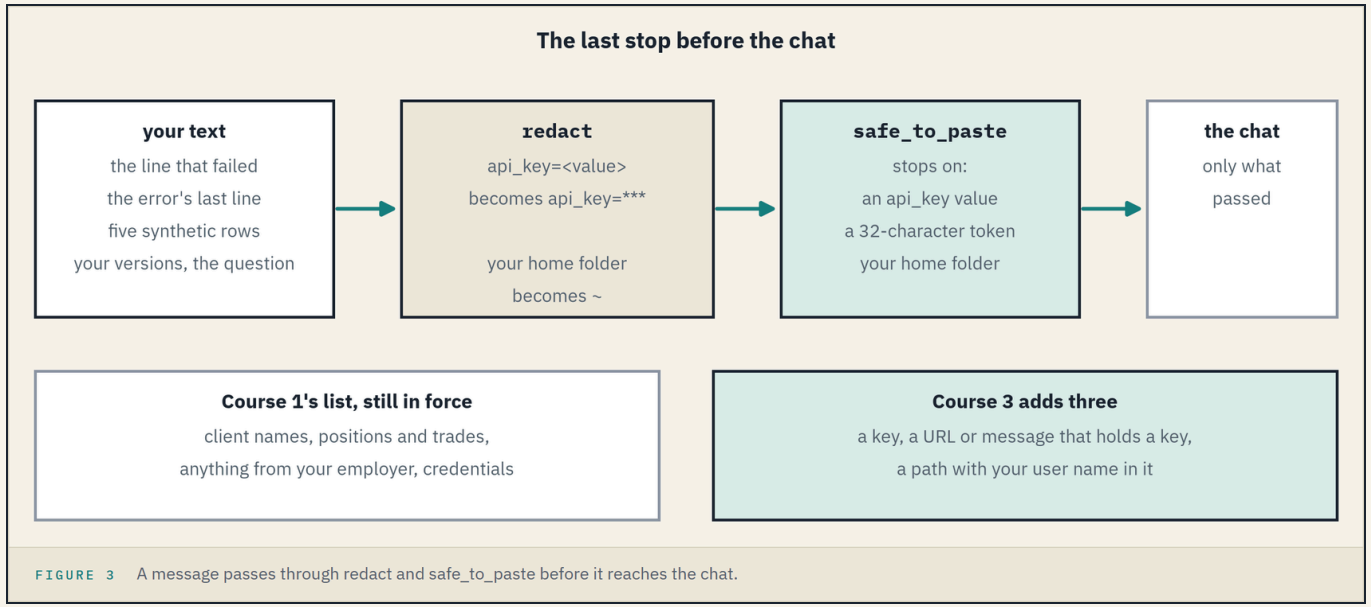


FIGURE 3 A message passes through redact and safe_to_paste before it reaches the chat.

THE MESSAGE AS IT CAME

```
safe_to_paste(http_message)

AssertionError: an api_key value
```

BUILT TO BE PASTED

```
safe_to_paste(paste)
print(paste)

This line fails:
md.change_bp(gappy, percent_units=False)
```

Paste the line, the error, five synthetic rows and the rule, run it through `safe_to_paste` first, and the key and your user name never leave your machine.

13.6 A Package That Does Not Exist

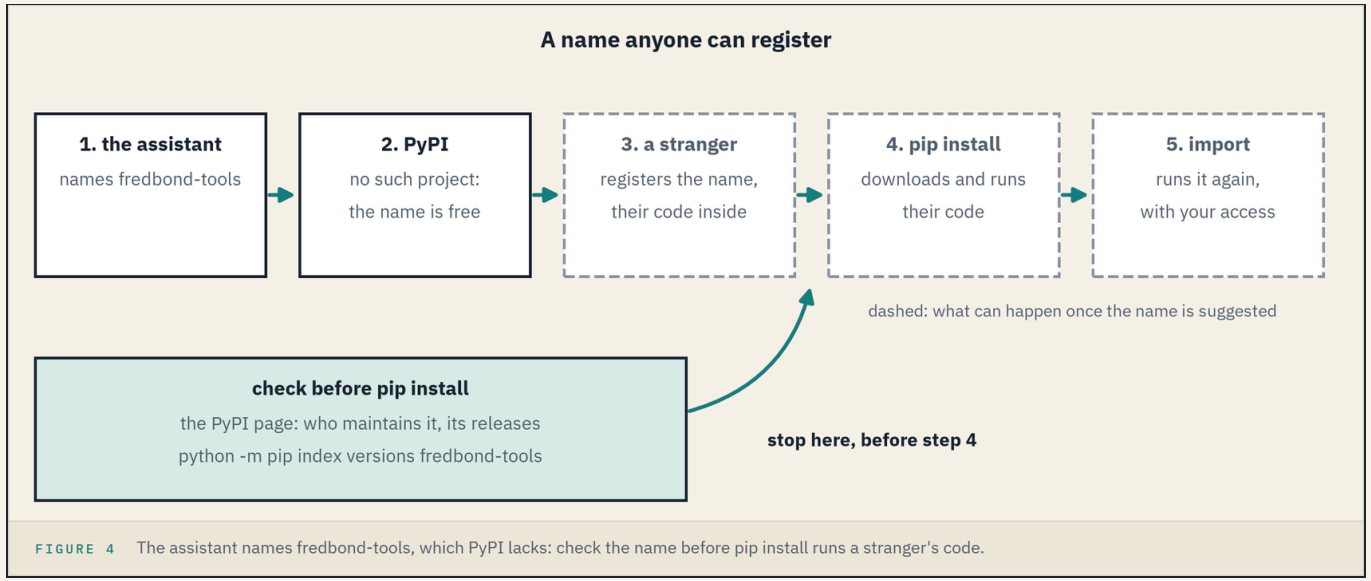


FIGURE 4 The assistant names fredbond-tools, which PyPI lacks: check the name before pip install runs a stranger's code.

THE IMPORT FAILS

```
import fredbond_tools

ModuleNotFoundError: No module named 'fredbond_tools'
```

CHECK THE NAME FIRST

```
# the PyPI page first: https://pypi.org/project/fredbond-tools/
python -m pip index versions fredbond-tools
python -m pip index versions requests
```

LOOK WITHOUT IMPORTING

```
fredbond_tools installed: False
pandas installed: True
```

Check a name on PyPI before installing it, and an argument with `inspect.signature` before trusting it: an assistant can invent both.

13.7 JSON Replies and parse_json_reply

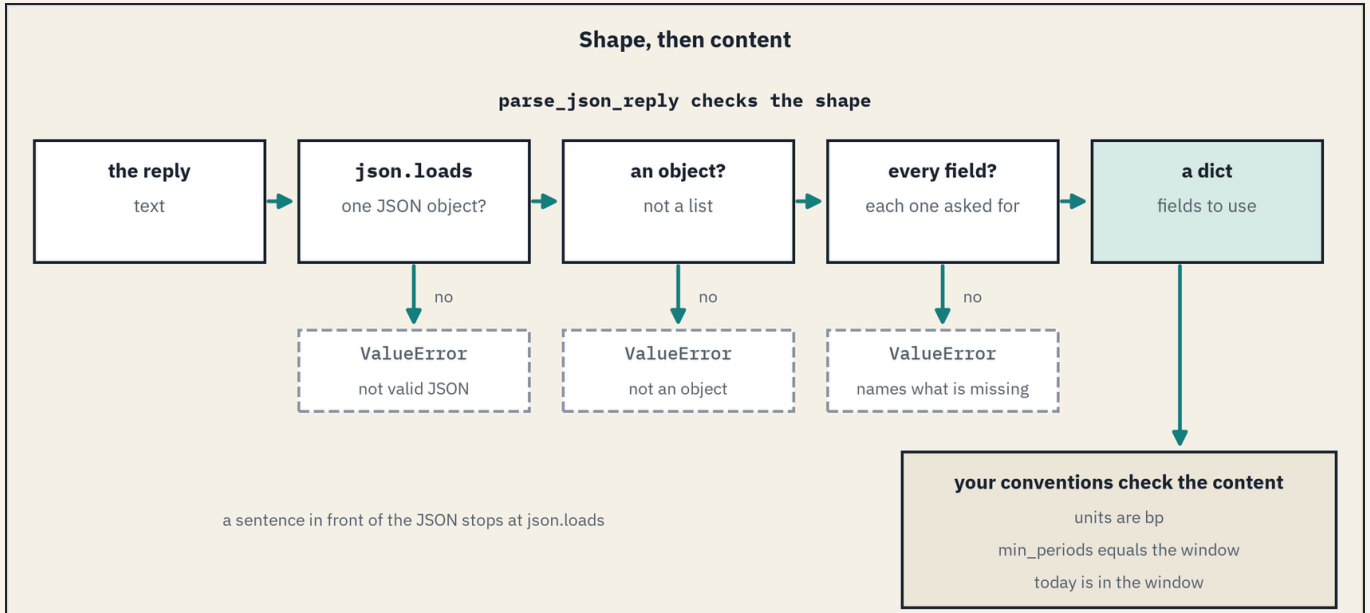


FIGURE 5 parse_json_reply checks the shape, then your conventions check the content: bp, min_periods, and the date.

THREE TESTS IT MUST PASS

```
def test_parse_json_reply_fields():
def test_parse_json_reply_missing_field_raises():
def test_parse_json_reply_not_json_raises():
```

A CLEAN REPLY

```
reply = md.parse_json_reply(replies['clean'], FIELDS)

DGS10: 63 rows, min_periods 63
```

`parse_json_reply` **checks the shape and your conventions check the content: both are tests, and both run on your machine, not in the assistant.**

13.8 An Assistant's Patch, Reviewed With Git

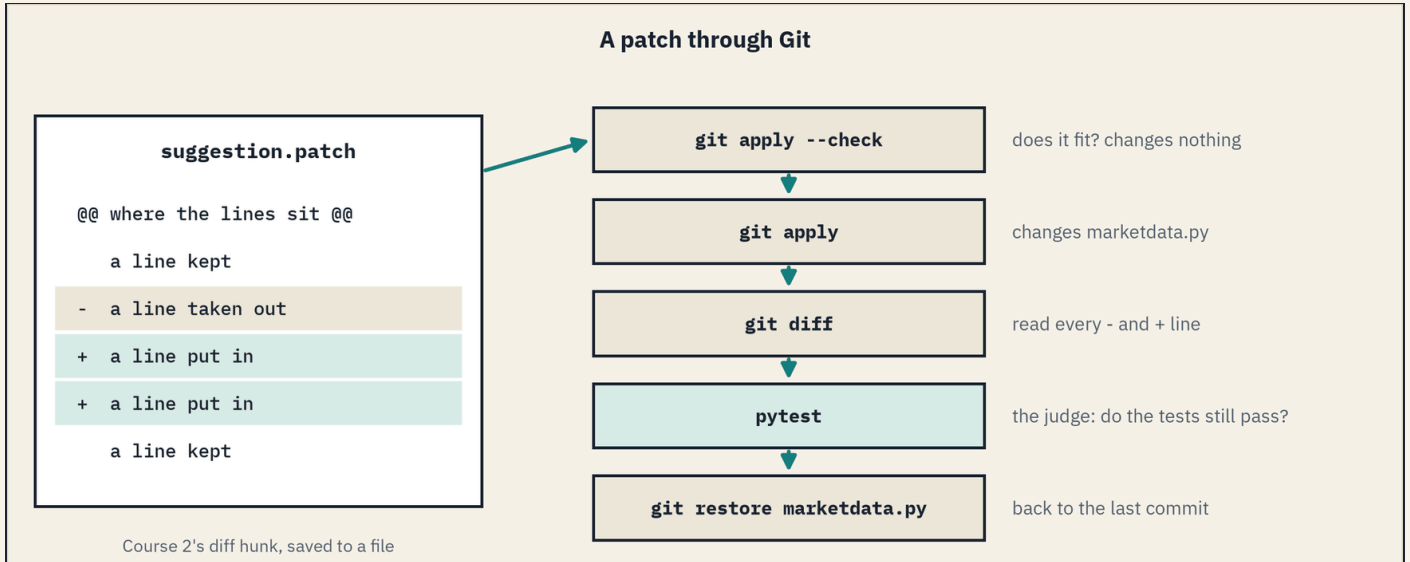


FIGURE 6 A patch is applied with a check, read with git diff, judged by pytest, and undone by git restore.

READ THE DIFF, COMMIT THE DAY

```
!git -C "{work}" status --short
!git -C "{work}" diff --stat

M marketdata.py
 M test_marketdata.py
 marketdata.py      | 25 ++++++++++++++++++++++++
 test_marketdata.py | 19 ++++++++++++++++++
 2 files changed, 44 insertions(+)
```

APPLY IT AND READ IT

```
!git -C "{work}" apply suggestion.patch
!git -C "{work}" diff
```

Two lines out, three in: the function now cuts the text from the first { to the last } before reading it.

Copy the work folder into my_bondmath.

Bring It Home

IN A POWERSHELL TERMINAL IN VS CODE

```
# the work folder: use the name your setup cell printed
$work = "$env:TEMP\pfi_course3_13_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py" .
python -m pytest -q test_marketdata.py
git diff --stat
git add marketdata.py test_marketdata.py
git commit -m "Notebook 13: parse_json_reply"
```

Two habits to keep: run `safe_to_paste` before you paste, and look a name up on PyPI before you install it.

You Can Now

- Write a prompt as a spec: the docstring, the conventions that apply, and a test.
- Run an assistant's code against your functions and Excel's values, at the start of a series too.
- Check a chart by reading its label and its line as data.
- Test an assistant's bond code against **`bondmath`** on dates where a mistake would show.
- Redact a key and your home folder, and check text with **`safe_to_paste`** before you paste it.
- Check a package name on PyPI and an argument with **`inspect.signature`**.
- Check a JSON reply's shape with **`parse_json_reply`** and its content with your conventions.
- Review a patch with **`git apply --check`**, **`git diff`** and the tests, and undo it with **`git restore`**.

NEXT
WEEK 3 · DAY 4

The Daily Market Snapshot

One page for one market date: the curve, two synthetic spreads, their shapes, and the book's DV01 with a what-if, written as HTML and as Excel by one command for any date, and never reading past it.